

AN SFTP

SFTP Service — Section 1: Service Brief

1.1 What is SFTP?

- **SFTP = SSH File Transfer Protocol**
 - Used for **secure file transfer** between a client and server.
 - Runs through **SSH**.
 - Supports:
 - Upload
 - Download
 - File/directory management
 - Normally uses **TCP port 22**.
 - Provides **encryption and authentication** through SSH.
-

1.2 Key Facts

Feature	SFTP
Full Name	SSH File Transfer Protocol
Architecture	Client–Server
Transport	SSH over TCP
Default Port	TCP 22
Encryption	Yes
Authentication	Password / SSH Key
File Operations	Upload / Download / Manage
Active/Passive Mode	Not used
Common Server	OpenSSH

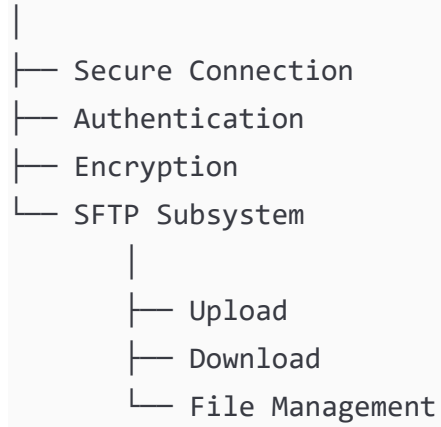
1.3 Why SFTP is Used

- Protects **credentials and file data** during transfer.
- Suitable for transferring **sensitive files** over untrusted networks.
- Provides secure remote file operations.

- Can **resume interrupted transfers** using SFTP client commands.
-

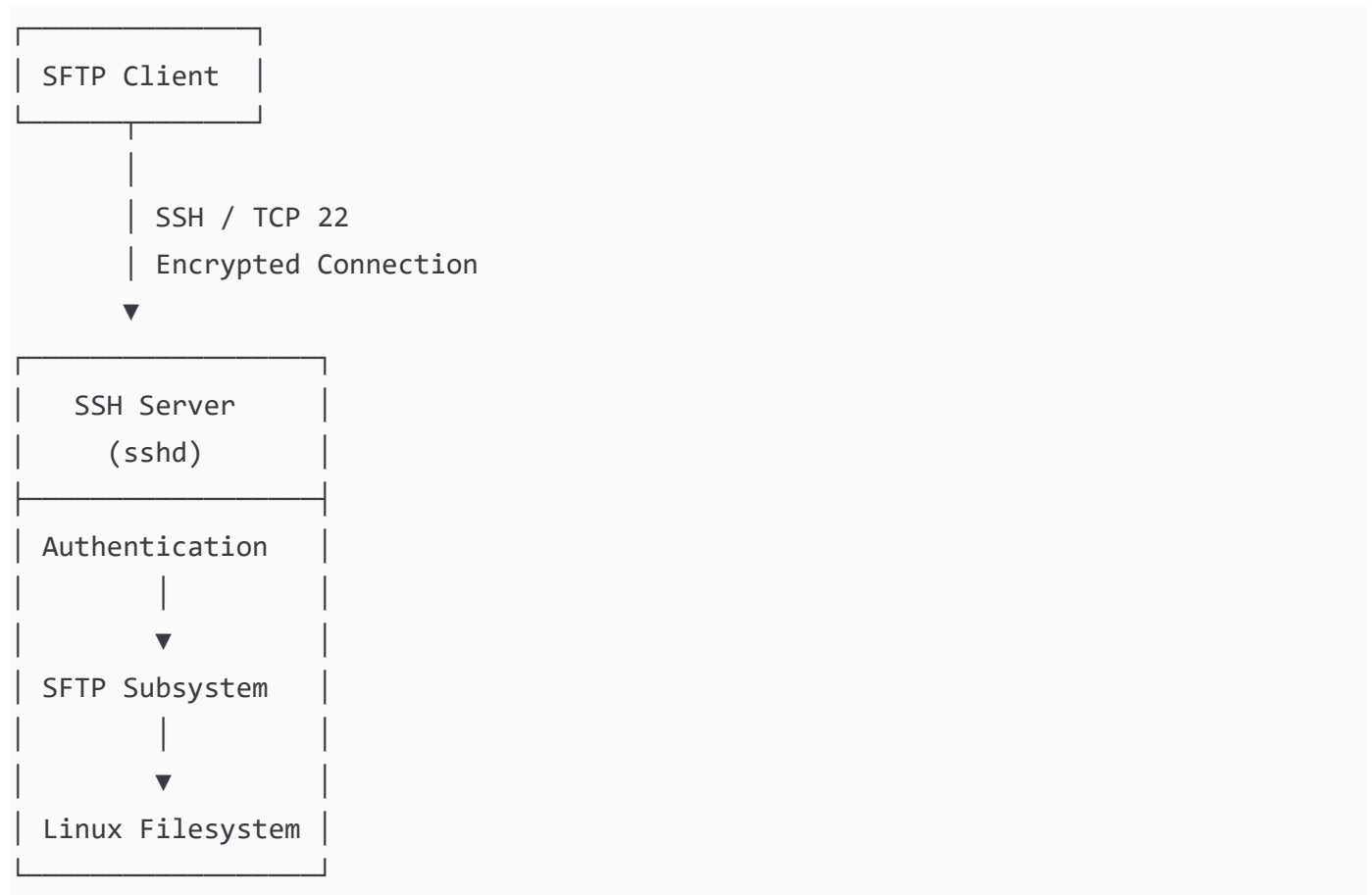
1.4 SFTP and SSH

SSH



Section 2: Architecture & Working

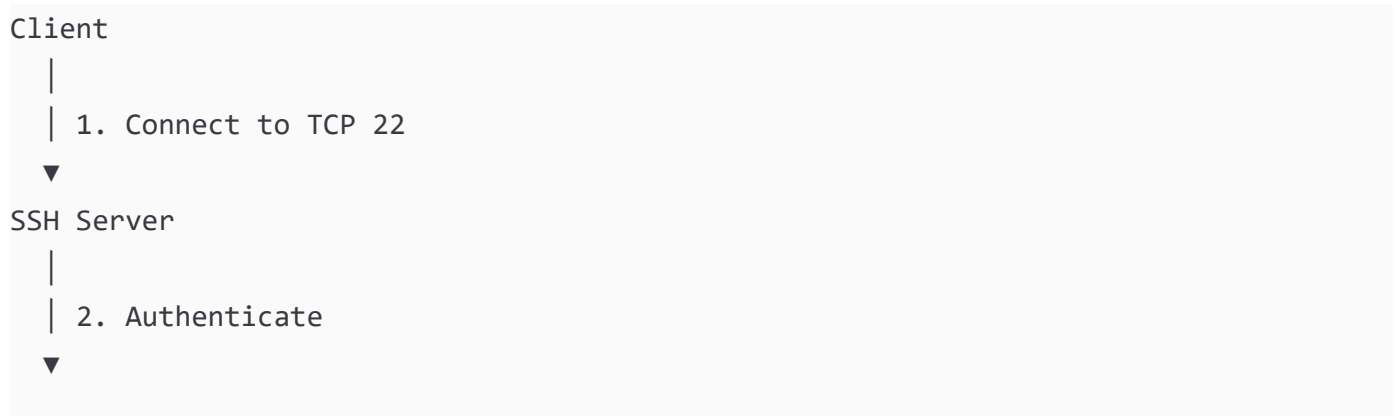
2.1 SFTP Architecture



Main Components

- **SFTP Client** — initiates the connection.
- **SSH Server (sshd)** — accepts and secures the connection.
- **Authentication** — verifies the user.
- **SFTP Subsystem** — handles file-transfer requests.
- **Linux Filesystem** — provides the actual files and directories.

2.2 Working Flow



SFTP Subsystem

```
|
| 3. Start SFTP Session
▼
```

Remote Filesystem

```
|
| └─ ls / pwd
| └─ cd
| └─ get  → Download
| └─ put  → Upload
```

Process

1. Client connects to the server on **TCP 22**.
2. SSH establishes the **secure encrypted connection**.
3. User is authenticated.
4. SSH starts the **SFTP subsystem**.
5. Client performs file and directory operations.
6. Transfer data and SFTP commands travel through the SSH-protected connection.
7. Session ends when the client disconnects.

2.3 Important Characteristics

- Uses **SSH client-server architecture**.
- Normally uses **TCP port 22**.
- Authentication and encryption are handled by SSH.
- SFTP operations run through the SSH connection.
- **No FTP control/data channel separation**.
- **No FTP Active/Passive modes**.
- Interrupted transfers can be resumed using commands such as `reget` and `reput`.

2.4 SFTP vs FTP

SFTP	FTP
Runs over SSH	Uses FTP protocol
TCP 22	TCP 21 + data connection
Encrypted	Plain FTP is unencrypted
Single SSH-based session	Separate control/data connections

SFTP	FTP
No Active/Passive modes	Active/Passive modes
SSH authentication	FTP authentication

Key Point: SFTP uses **SSH + SFTP**, while FTP uses the traditional **FTP control and data connection model**.

SFTP Service — Section 3: Authentication & Access Models

3.1 Authentication Methods

SFTP uses SSH authentication:

Method	Description
Password	Username and password
SSH Key	Public/private key pair

Password Authentication

```
sftp sftpuser@192.168.1.10
```

- Simple to configure.
- Suitable for basic users and lab environments.

SSH Key Authentication

Generate a key pair on the client:

```
ssh-keygen
```

Copy the public key to the server:

```
ssh-copy-id sftpuser@192.168.1.10
```

Connect:

```
sftp sftpuser@192.168.1.10
```

- More secure than password-only authentication.
- Useful for administrators and automated file transfers.
- **Private key remains on the client.**

3.2 Linux Access Control

SFTP follows normal Linux filesystem permissions.

Access can be controlled using:

- Users
- Groups
- Ownership

- Permissions
- ACLs

Example:

```
ls -ld /srv/sftp/shared
```

```
sudo chown sftpuser:sftpusers /srv/sftp/shared
```

```
sudo chmod 770 /srv/sftp/shared
```

SFTP does not bypass Linux filesystem permissions.

3.3 SFTP-Only Users

A user can be allowed to transfer files without receiving a normal SSH shell.

Configure in:

```
/etc/ssh/sshd_config
```

Example:

```
Match User sftpuser
```

```
    ForceCommand internal-sftp
```

What does `ForceCommand` mean?

Normally:

```
SSH → Authentication → Linux Shell
```

With:

```
ForceCommand internal-sftp
```

the user is forced into:

```
SSH → Authentication → SFTP
```

Therefore:

- SFTP file operations → Allowed
- Normal SSH shell → Not provided
- Useful for dedicated file-transfer accounts

`ForceCommand` controls WHAT service the user can use.

3.4 Chroot Access

Chroot restricts an SFTP user's filesystem environment to a specified directory.

Example directory structure:

```
/
├─ sftp/
│   └─ sftpuser/
│       └─ upload/
```

Configure:

```
Match User sftpuser
    ForceCommand internal-sftp
    ChrootDirectory /sftp/%u
```

3.5 Understanding ChrootDirectory

The following configuration:

```
ChrootDirectory /sftp/%u
```

means:

Make `/sftp/<username>` the filesystem root for that SFTP user.

For `sftpuser`:

```
%u = sftpuser
```

Therefore:

```
/sftp/%u
```

becomes:

```
/sftp/sftpuser
```

What is `%u`?

`%u` is an SSH configuration token representing the **username of the user who is connecting**.

Connecting User	<code>%u</code>	Chroot Directory
sftpuser	sftpuser	<code>/sftp/sftpuser</code>
backupuser	backupuser	<code>/sftp/backupuser</code>

Connecting User	%u	Chroot Directory
financeuser	financeuser	/sftp/financeuser

This allows one configuration to support multiple users.

3.6 Actual Filesystem vs User's View

Suppose the server has:

```
Server Filesystem
/
├─ etc/
├─ home/
├─ var/
└─ sftp/
    └─ sftpuser/
        └─ upload/
```

After chroot:

```
SFTP User View
/
└─ upload/
```

The mapping is:

Server Path	SFTP User Sees
/sftp/sftpuser/	/
/sftp/sftpuser/upload/	/upload/

So when the user runs:

```
sftp> pwd
```

they may see:

```
Remote working directory: /
```

But the actual server location is:

```
/sftp/sftpuser/
```

If the user uploads:

```
sftp> cd upload
sftp> put test.txt
```

the actual file is stored at:

```
/sftp/sftpuser/upload/test.txt
```

3.7 Chroot Directory Ownership

The chroot root directory must be controlled by **root**.

Create the structure:

```
sudo mkdir -p /sftp/sftpuser/upload
```

Set ownership:

```
sudo chown root:root /sftp/sftpuser
sudo chmod 755 /sftp/sftpuser
```

Give the user a writable subdirectory:

```
sudo chown sftpuser:sftpuser /sftp/sftpuser/upload
sudo chmod 755 /sftp/sftpuser/upload
```

Final structure:

```
/sftp/sftpuser/          ← root:root
|
└─ upload/              ← sftpuser:sftpuser
```

Why a separate `upload` directory?

The chroot root itself should not be writable by the SFTP user.

Therefore:

- `/sftp/sftpuser/` → **root-owned**
- `/sftp/sftpuser/upload/` → **user-writable**

This provides the required security boundary while still allowing file uploads.

3.8 Why Chroot Restricts Access

Without chroot, the server filesystem may contain:

```
/
├─ etc/
├─ home/
├─ var/
├─ usr/
└─ sftp/
```

With:

```
ChrootDirectory /sftp/sftpuser
```

the user's filesystem starts at:

```
/sftp/sftpuser/
```

which appears to the user as:

```
/
```

Therefore, the user cannot use the SFTP session to navigate to the server's actual:

```
/etc
/home
/var
/usr
```

unless those paths are separately available inside the chroot environment.

3.9 Combining SFTP-Only + Chroot

A common restricted SFTP configuration is:

```
Match User sftpuser
    ForceCommand internal-sftp
    ChrootDirectory /sftp/%u
    AllowTcpForwarding no
    X11Forwarding no
```

The four important ideas are:

Configuration	Meaning
<code>Match User sftpuser</code>	WHO — applies to this user
<code>ForceCommand internal-sftp</code>	WHAT — SFTP only
<code>ChrootDirectory /sftp/%u</code>	WHERE — restricted filesystem

Configuration	Meaning
<code>%u</code>	WHICH USER — connecting username

Easy way to remember

`Match User` → WHO

`ForceCommand` → WHAT

`ChrootDirectory` → WHERE

`%u` → USERNAME

This combination is commonly used for accounts that need **secure file transfer without general SSH access**.

Section 4: Prerequisites & Installation

4.1 Requirements

- Linux server with **OpenSSH Server**
 - SFTP-capable client
 - Network connectivity between client and server
 - **TCP port 22** accessible
 - Valid SFTP user account
 - Correct file/directory permissions
-

4.2 Install OpenSSH Server

On Debian/Ubuntu:

```
sudo apt update
sudo apt install openssh-server
```

4.3 Start and Enable SSH

```
sudo systemctl start ssh
sudo systemctl enable ssh
```

Check service status:

```
sudo systemctl status ssh
```

Expected:

```
Active: active (running)
```

4.4 Verify TCP Port 22

```
sudo ss -lntp | grep :22
```

Expected:

```
LISTEN ... :22 ... sshd
```

If a firewall is enabled:

```
sudo ufw allow 22/tcp
```

4.5 Test SFTP Connection

From the client:

```
sftp username@SERVER_IP
```

Example:

```
sftp sftpuser@192.168.1.10
```

After successful authentication:

```
sftp>
```

This confirms that the SFTP service is installed, running, and reachable.

4.6 Important Resources

Resource	Purpose
/etc/ssh/sshd_config	SSH/SFTP configuration
/home/<username>/	Normal user directory
/sftp/<username>/	Example chroot directory
TCP 22	SFTP/SSH connection

Key Point: SFTP is provided through **OpenSSH**, normally using **TCP port 22**.

Section 5: Server Configuration

5.1 Main Configuration File

SFTP is configured through the OpenSSH configuration file:

```
/etc/ssh/sshd_config
```

Before making changes, create a backup:

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
```

5.2 Basic SFTP Configuration

Open the configuration:

```
sudo nano /etc/ssh/sshd_config
```

Make sure the SFTP subsystem is enabled.

For OpenSSH, either of these configurations may be used:

```
Subsystem sftp internal-sftp
```

or:

```
Subsystem sftp /usr/lib/openssh/sftp-server
```

`internal-sftp` runs the SFTP subsystem directly inside `sshd`, while `sftp-server` is a separate SFTP server program.

5.3 Configure an SFTP-Only User

For a dedicated SFTP user:

```
Match User sftpuser
    ForceCommand internal-sftp
    ChrootDirectory /sftp/%u
    AllowTcpForwarding no
    X11Forwarding no
```

Purpose

Directive	Purpose
<code>Match User sftpuser</code>	Applies settings to this user
<code>ForceCommand internal-sftp</code>	Allows SFTP instead of shell
<code>ChrootDirectory /sftp/%u</code>	Restricts filesystem access
<code>%u</code>	Connecting username
<code>AllowTcpForwarding no</code>	Disables TCP forwarding
<code>X11Forwarding no</code>	Disables X11 forwarding

5.4 Validate Configuration

Always check the configuration before restarting SSH:

```
sudo sshd -t
```

If there is no output, the configuration is valid.

Restart SSH:

```
sudo systemctl restart ssh
```

Check:

```
sudo systemctl status ssh
```

Key Point: Make configuration changes in `/etc/ssh/sshd_config`, validate with `sshd -t`, and only then restart the SSH service.

Section 6: Practical SFTP Setup

6.1 Create User and Directory

Create a dedicated SFTP user:

```
sudo adduser sftpuser
```

Create the chroot and writable directory:

```
sudo mkdir -p /sftp/sftpuser/upload
```

Set ownership and permissions:

```
sudo chown root:root /sftp/sftpuser
```

```
sudo chmod 755 /sftp/sftpuser
```

```
sudo chown sftpuser:sftpuser /sftp/sftpuser/upload
```

```
sudo chmod 755 /sftp/sftpuser/upload
```

Directory structure:

/sftp/sftpuser/	→ Chroot root
└─ upload/	→ User-writable directory

6.2 Test SFTP Connection

From the client:

```
sftp sftpuser@SERVER_IP
```

Example:

```
sftp sftpuser@192.168.1.10
```

Check the remote location:

```
sftp> pwd
```

```
sftp> ls
```

Expected:

```
Remote working directory: /
```

6.3 Test File Transfer

Upload a file:

```
sftp> cd upload  
sftp> put test.txt  
sftp> ls
```

Download the file:

```
sftp> get test.txt
```

Exit the session:

```
sftp> bye
```

6.4 Verify on Server

Check the uploaded file:

```
ls -l /sftp/sftpuser/upload/
```

Test SFTP-only restriction:

```
ssh sftpuser@192.168.1.10
```

The user should not receive a normal interactive shell.

Key Point: The practical setup creates the SFTP user's directory structure, applies the required permissions, and verifies connection and file transfer.

Section 7: SFTP Client Operations

7.1 Connect to Server

```
sftp username@SERVER_IP
```

Example:

```
sftp sftpuser@192.168.1.10
```

Using a custom SSH port:

```
sftp -P 2222 sftpuser@192.168.1.10
```

Using an SSH key:

```
sftp -i ~/.ssh/id_rsa sftpuser@192.168.1.10
```

7.2 Essential Commands

Command	Purpose
<code>pwd</code>	Show remote directory
<code>lpwd</code>	Show local directory
<code>ls</code>	List remote files
<code>cd <dir></code>	Change remote directory
<code>lcd <dir></code>	Change local directory
<code>get <file></code>	Download
<code>put <file></code>	Upload
<code>mkdir <dir></code>	Create remote directory
<code>rename old new</code>	Rename file
<code>reget <file></code>	Resume download
<code>reput <file></code>	Resume upload
<code>bye</code>	Exit SFTP

7.3 Basic File Transfer

Upload:

```
sftp> put test.txt
```

Download:

```
sftp> get report.pdf
```

Resume interrupted transfer:

```
sftp> reput test.txt  
sftp> reget report.pdf
```

Key Point: `cd`, `ls`, and `pwd` operate on the **remote server**, while `lcd` and `lpwd` operate on the **local client**.

Section 8: Security & Troubleshooting

8.1 Security Practices

- Prefer **SSH key authentication** for administrative and automated transfers.
 - Use **strong passwords** when password authentication is enabled.
 - Disable **root SSH login** where appropriate.
 - Keep **OpenSSH updated** with security patches.
 - Restrict TCP 22 access using firewall rules when possible.
 - Use dedicated **SFTP-only users** for file-transfer access.
 - Use **ChrootDirectory** when users must be isolated from the wider filesystem.
 - Apply the **least-privilege** principle to files and directories.
-

8.2 SFTP User Restrictions

For restricted users:

```
ForceCommand internal-sftp
ChrootDirectory /sftp/%u
AllowTcpForwarding no
X11Forwarding no
```

Provides:

- SFTP access without a normal shell
 - Restricted filesystem environment
 - Reduced unnecessary SSH functionality
-

8.3 File Permissions

Check permissions:

```
ls -ld /sftp/sftpuser/upload/
```

Correct ownership:

```
sudo chown sftpuser:sftpuser /sftp/sftpuser/upload
```

Avoid unnecessary write permissions.

8.4 Troubleshooting

Common Issues

Problem	Check / Command
SSH service down	<code>sudo systemctl status ssh</code>
Port 22 unavailable	<code>sudo ss -lntp grep :22</code>
Login failure	Check user, password/key, account status
Permission denied	<code>ls -ld /sftp/sftpuser/upload/</code>
Chroot failure	Check chroot ownership: <code>root:root</code>
Configuration error	<code>sudo sshd -t</code>
Need logs	<code>sudo journalctl -u ssh</code>

Troubleshooting Flow

Service → Port → Authentication → Permissions → Chroot → Logs

Key Point: Always validate `sshd_config` with `sshd -t` before restarting SSH.

Section 9: Real-World Scenario

9.1 Secure Vendor File Transfer

A company needs to exchange files securely with an external vendor.

Requirements:

- Secure upload/download
- No SSH shell access
- No access to other server directories

Solution:

Vendor

```
|  
| SFTP / TCP 22
```



SFTP Server

```
|  
└─ /sftp/vendor1/  
    └─ upload/
```

Configuration:

Match User vendor1

```
ForceCommand internal-sftp  
ChrootDirectory /sftp/%u
```

Result:

- Encrypted file transfer
- SFTP-only access
- Restricted filesystem
- Controlled file access

Section 10: Viva / Interview Questions

1. **What is SFTP?**

SSH File Transfer Protocol for secure file transfer.

2. **Which port does SFTP use?**

TCP **22** by default.

3. **SFTP vs FTPS?**

SFTP uses SSH; FTPS uses FTP with TLS.

4. **Does SFTP use Active/Passive modes?**

No.

5. **Which Linux service provides SFTP?**

OpenSSH (`sshd`).

6. **SFTP configuration file?**

`/etc/ssh/sshd_config`

7. **What does `ForceCommand internal-sftp` do?**

Forces SFTP instead of a normal SSH shell.

8. **What does `ChrootDirectory` do?**

Restricts the user's filesystem environment.

9. **What does `%u` mean?**

The connecting username.

10. **How do you test SSH configuration?**

```
sudo sshd -t
```

11. **How do you check the SSH service?**

```
sudo systemctl status ssh
```

Section 11: Quick Reference

Important

Item	Value
Protocol	SSH File Transfer Protocol
Service	OpenSSH (<code>sshd</code>)
Default Port	TCP 22
Configuration	<code>/etc/ssh/sshd_config</code>
Example Chroot	<code>/sftp/<user>/</code>

Service Commands

```
sudo systemctl restart ssh
sudo systemctl status ssh
sudo systemctl enable ssh
```

Configuration & Port Check

```
sudo sshd -t
sudo ss -lntp | grep :22
```

Client Connection

```
sftp user@SERVER_IP
sftp -P 2222 user@SERVER_IP
sftp -i ~/.ssh/id_rsa user@SERVER_IP
```

Essential SFTP Commands

Command	Function
<code>ls</code>	List remote files
<code>pwd</code>	Remote directory
<code>cd</code>	Change remote directory
<code>lpwd</code>	Local directory
<code>lcd</code>	Change local directory
<code>put</code>	Upload
<code>get</code>	Download

Command	Function
<code>reput</code>	Resume upload
<code>reget</code>	Resume download
<code>mkdir</code>	Create directory
<code>rename</code>	Rename file
<code>bye</code>	Exit

Restricted SFTP

```
Match User sftpuser
    ForceCommand internal-sftp
    ChrootDirectory /sftp/%u
```

SFTP = SSH-based secure file transfer with Linux authentication, permissions, and optional user isolation.